

PCPT Notebooks: A Preparation Course for PyTorch with a Focus on Signal Processing

Meinard Müller¹, Johannes Zeitler¹, and Sebastian Strahl¹

¹ International Audio Laboratories Erlangen, Germany

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Submitted: 08 December 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

PyTorch¹ (Paszke et al., 2019) is among the most widely used machine-learning frameworks in research and education, supported by a vibrant ecosystem of tutorials, textbooks, and courses, as reflected by curated collections such as *The Incredible PyTorch*². The Preparation Course for PyTorch (PCPT) notebooks, which we present here, are not intended to replace these offerings; rather, they provide a complementary, low-barrier route into PyTorch for learners with basic Python experience. PCPT builds directly on our PCP notebooks (Müller & Rosenzweig, 2022) and is conceived as a follow-up: assuming familiarity with core Python concepts, it introduces PyTorch and fundamental machine-learning ideas in a structured and accessible way. Built on the interactive Jupyter framework (Kluyver et al., 2016), the notebooks combine executable code, textbook-style explanations, mathematical derivations, and visualizations in a single learning environment.

PCPT is more than a programming course. It revisits and deepens essential concepts from both machine learning and signal processing, including gradient descent, linear regression, discrete convolution, recursion, binary and multiclass classification, loss functions, and common evaluation metrics. A distinctive aspect is the tight connection to digital signal processing (DSP) in the audio domain: machine-learning ideas are consistently related to convolutional and recursive filters, low- and high-pass filtering, smoothing, denoising, and onset detection, with an emphasis on one-dimensional sequence data rather than two-dimensional image examples. PCPT is suitable for self-study and teaching, particularly for students at the end of their Bachelor's studies or the beginning of a Master's program. To keep entry barriers low, the notebooks rely on toy problems and synthetic datasets, focus on principles instead of large-scale data, and run efficiently on standard CPUs without specialized hardware.

Structure, Content, and Access

The PCPT course is organized in a modular fashion and consists of ten units, each corresponding to an individual notebook. The structure and content of the course are illustrated in Figure 1, which is also part of the starting notebook of the PCPT course. Broadly speaking, the first part of PCPT introduces central PyTorch concepts and machine-learning fundamentals, while the second part deepens these notions through applications that are closely connected to DSP. In particular, the PCPT notebooks are designed as a follow-up to our PCP notebooks and are deliberately similar in structure, didactic style, and learning goals, now shifting the focus from Python basics to PyTorch and machine learning.

¹<https://pytorch.org/>

²<https://github.com/ritchieng/the-incredible-pytorch>

Unit	Title	Notions, Techniques & Algorithms	HTML	IPYNB
1	Get Started	Download; Conda; Python environment; Jupyter	[html]	[ipynb]
2	Python Classes	class definition; object-oriented programming; <code>__init__()</code> ; instance attributes; class attributes; <code>__doc__</code> ; methods; <code>self</code> ; inheritance; <code>super()</code> ; overriding methods	[html]	[ipynb]
3	PyTorch Tensors	Tensor creation; <code>torch.tensor</code> ; tensor attributes; <code>.shape</code> ; data types; tensor operations; indexing; slicing; reshaping; <code>.squeeze</code> ; linear algebra; eigen decomposition; storage; memory layout	[html]	[ipynb]
4	Automatic Differentiation	Gradient computation; gradient descent; <code>.grad</code> ; automatic differentiation; <code>torch.autograd</code> ; <code>requires_grad</code> ; backpropagation; <code>.backward()</code> ; parameter update; tensor shapes; batch dimension; polynomial regression	[html]	[ipynb]
5	Neural Network Modules	Neural networks; layers; <code>torch.nn</code> ; <code>nn.module</code> ; linear model; <code>nn.linear</code> ; non-linear model; activation functions; <code>ReLU</code> ; <code>sigmoid</code> ; <code>tanh</code> ; loss function; <code>nn.MSELoss</code> ; optimizers; <code>optim.SGD</code> ; <code>torch.optim</code> ; training loop; polynomial regression; piecewise linear approximation	[html]	[ipynb]
6	Convolution and CNNs	Discrete 1D convolution; cross-correlation; <code>nn.Conv1d</code> ; kernel size; padding; stride; dilation; denoising; smoothing; edge detection; low-pass filter; high-pass filter; band-pass filter; <code>Dataset</code> ; <code>DataLoader</code>	[html]	[ipynb]
7	Classification	Binary classification; multiclass classification; sigmoid; softmax; logits; cross-entropy loss; <code>nn.CrossEntropyLoss</code> ; <code>nn.Sequential</code> ; sine waves; square waves; frequency; accuracy; confusion matrix; out-of-distribution data	[html]	[ipynb]
8	Training Dynamics	Overfitting; generalization; 2D point cloud classification; training set; validation set; test set; validation monitoring; loss and accuracy curves; early stopping; learning rate scheduling	[html]	[ipynb]
9	Recursion and RNNs	Recursive filters; FIR/IIR; impulse response; recurrent neural networks; <code>nn.RNN</code> ; hidden states; onset detection; class imbalance; peak picking; precision; recall; F-measure	[html]	[ipynb]
10	Further Essential Techniques	Broadcasting; gradient flow; detach; input normalization; batch normalization; regularization; weight decay; L2 regularization; dropout; custom functions; L1 loss; correlation loss; spectral loss	[html]	[ipynb]

Figure 1: Overview of the PCPT notebooks' ten units and their main content.

After a short notebook on how to get started (Unit 1), the course begins with a focused recap of Python classes and object-oriented concepts needed to read and build PyTorch models (Unit 2). It then introduces PyTorch tensors as the core data structure (Unit 3), covering creation, shapes, indexing, reshaping, and basic linear-algebra operations. On this basis, automatic differentiation and gradient-based optimization are explained (Unit 4), enabling students to construct and train neural networks in PyTorch, making use of linear layers, activation functions, loss functions, and optimizers (Unit 5).

With the PyTorch toolkit in place, the course explicitly connects machine-learning concepts to DSP. Discrete one-dimensional convolution and convolutional neural networks (CNNs) are introduced as learnable counterparts of classical DSP filtering, demonstrated through smoothing, denoising, and low-pass or high-pass examples (Unit 6). These ideas are then carried into supervised learning for signals (Unit 7), with applications such as distinguishing noisy sines from square waves and classifying waveforms according to predefined frequency bands, and with evaluation via accuracy, confusion matrices, and out-of-distribution tests; exercises cover binary cross-entropy, softmax with cross-entropy, and frequency band classification using frequency-domain features. The course then turns to training dynamics, focusing on overfitting, generalization, and validation strategies to support reliable model

development (Unit 8).

Recursion is revisited by starting from intuitive classical recursive digital filters and impulse responses, then showing how recurrent neural networks (RNNs) realize learned recursive systems in PyTorch (Unit 9). This link is made concrete through an onset-detection case study with peak picking and evaluation by precision, recall, and F-measure, complemented by exercises on recursive filtering and class-imbalance handling. The course concludes with further essential PyTorch techniques (Unit 10), including broadcasting, normalization, regularization, gradient-flow considerations, and custom loss functions. Across all units, PyTorch ideas are grounded in DSP perspectives such as convolutional and recursive filters, spectral and temporal smoothing, denoising, and event detection, with a focus on one-dimensional sequence data suited to audio and signal-processing tasks.

Each unit is organized in a similar fashion. After giving an overview of a unit's structure and learning objectives, the actual content is presented using executable code, textbook-like explanations, mathematical formulas, plots, and visualizations. At the end of each unit, one finds short coding exercises. For self-evaluation, the PCPT notebooks provide for each exercise a sample solution in the form of a Python function, contained in one of the modules of the Python package `libpcpt`. Each such function, which can be easily traced back by its function name, is executed after the respective exercise to produce the results in question.

The PCPT notebooks (including the text, code, and figures) are licensed under the open-source MIT License. Our goal is to continuously improve the PCPT notebooks and provide updates on a regular basis (current version: 1.0.0). To keep the initial hurdles as low as possible and to account for different user needs, the PCPT notebooks can be accessed and executed in different ways.

- The primary source of the PCPT material is the GitHub repository ³. It contains executed versions of all notebooks and all source code.
- Additionally, exported HTML versions of all PCPT notebooks can be found at the authors' institutional website ⁴. This static version allows users to access all material, including the explanations, figures, and code examples, by just following the HTML links.
- To execute the Python code cells, one needs to download the notebooks, install all required Python packages, and start a Jupyter server. The necessary steps are explained in detail in the PCPT notebook on how to get started (Unit 1). Within a Jupyter session, one can follow the IPYNB links for navigating between the units.
- As an alternative to running the notebooks locally, one can also use web-based services such as Google Colab ⁵ and Binder ⁶. Explanations on how to run the PCPT notebooks using these services can be found in the notebooks' GitHub repository.

Educational Considerations

The immediate educational contribution of the PCPT notebooks is to provide a structured and concept-focused entry into PyTorch and modern machine learning for students who already possess basic Python programming skills, such as those acquired through our PCP notebooks. The PCPT notebooks are not intended to be a comprehensive PyTorch reference, nor do they aim to cover large-scale training or industrial deployment. Instead, they focus on a compact set of core ideas and deepen them through lightweight and transparent examples that run on standard CPUs. This low-barrier setup allows students to

³<https://github.com/meinardmueller/PCPT>

⁴<https://audiolabs-erlangen.de/PCPT>

⁵<https://colab.research.google.com>

⁶<https://mybinder.org>

101 concentrate on understanding principles and developing intuition without being distracted
102 by technical overhead.

103 A central design goal is continuity with the PCP notebooks. While PCP introduces
104 Python basics and fundamental DSP topics, PCPT builds on this foundation and revisits
105 familiar signal-processing concepts through the lens of learning-based models. For example,
106 discrete convolution is treated both as a classical DSP operation and as a building block
107 of CNNs, while recursion is explored in terms of recursive DSP filters and RNNs. Classical
108 tasks such as smoothing, denoising, and onset detection serve as tangible application
109 scenarios for training and evaluation. In this way, PCPT complements core courses in
110 signal processing and in multimedia, audio, and music processing, and is particularly well
111 suited for students at the end of their Bachelor's studies or the beginning of a Master's
112 program.

113 Pedagogically, the notebooks alternate systematically between concise theory and hands-
114 on practice. Key mathematical background is summarized in dedicated text boxes that
115 highlight definitions, essential formulas, and take-away messages. These compact textbook-
116 style elements provide a clear theoretical anchor, which is then immediately connected
117 to executable code, small experiments, and guided exercises. This structure allows
118 learners to engage with the material at different depths and supports both self-study and
119 classroom teaching. In our own practice, we use PCPT as a basis for practical courses and
120 research internships in international Master's programs spanning Multimedia Engineering,
121 Computer Science, Data Science, and Artificial Intelligence. In these settings, students
122 work through the units at different speeds, while tutors provide individualized support
123 and encourage collaboration.

124 The PCPT notebooks are designed to complement existing open educational resources in
125 a targeted way. They extend this programming foundation from basic Python to PyTorch
126 and machine learning. With respect to digital signal processing, they pair naturally with
127 McFee's *Digital Signals Theory* (McFee, 2023), which provides an accessible theoretical
128 grounding in discrete-time signals, convolution, and filtering. These concepts are revisited
129 in PCPT from a learning-based perspective. For audio and music processing, PCPT
130 bridges to our FMP notebooks (Müller & Zalkow, 2019) and the textbook *Fundamentals*
131 *of Music Processing* (Müller, 2021), preparing students to apply machine-learning methods
132 to concrete audio and music tasks. In this broader ecosystem, widely used toolboxes
133 such as librosa (McFee et al., 2015) offer standardized reference implementations that
134 students can adopt once they have acquired the necessary programming and machine-
135 learning foundations. More broadly, the PCPT notebooks support the objectives of open
136 and reproducible research, in line with recommendations for transparent and sustainable
137 software practices in music signal processing (McFee et al., 2019; Müller et al., 2021).

138 Finally, by working within the Jupyter environment, students become familiar with modern
139 workflows for scientific computing and reproducible experimentation. Together with the
140 course's modular organization, integrated solutions via libpcpt, and lightweight execution
141 on standard hardware, this makes PCPT a flexible teaching component that can be
142 embedded into signal-processing curricula, machine-learning modules, or research-oriented
143 lab and internship settings.

144 Statement of Need

145 The PCPT notebooks provide open-source educational material for an accessible entry into
146 PyTorch and machine learning, explicitly positioned as a follow-up to our PCP notebooks.
147 Complementing existing tutorials, PCPT focuses on a small set of foundational concepts
148 in machine learning and reinforces them through interactive coding, clear explanations,
149 and principled exercises.

150 A key value of PCPT lies in its close connection to one-dimensional signal processing.

Several introductory resources use image-based examples that require two-dimensional data handling and extensive preprocessing. In contrast, PCPT emphasizes 1D sequence data and DSP-motivated tasks, which are often more transparent and technically lightweight, especially in audio-related curricula.

Organized into ten compact units with integrated solutions and the supporting package `libpcpt`, the notebooks can be used flexibly in preparatory courses, internships, or for self-study. In summary, we hope that PCPT guides students from basic Python knowledge to practical PyTorch proficiency, and that it supports their move toward independent research in machine learning.

Acknowledgements

The PCPT notebooks are based on material developed by the authors and were refined through valuable feedback from students who used the course in practice. We gratefully acknowledge this feedback, which helped improve the notebooks. This work was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Grant No. 500643750 (MU 2686/15-1). The International Audio Laboratories Erlangen are a joint institution of the Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU) and the Fraunhofer Institute for Integrated Circuits IIS.

References

- Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & team, J. development. (2016). Jupyter notebooks—a publishing format for reproducible computational workflows. *Proceedings of the International Conference on Electronic Publishing*, 87–90.
- McFee, B. (2023). *Digital signals theory* (1st ed., pp. 1–277) [Monograph]. Chapman; Hall/CRC. <https://doi.org/10.1201/9781003264859>
- McFee, B., Kim, J. W., Cartwright, M., Salamon, J., Bittner, R. M., & Bello, J. P. (2019). Open-source practices for music signal processing research: Recommendations for transparent, sustainable, and reproducible audio research. *IEEE Signal Processing Magazine*, 36(1), 128–137. <https://doi.org/10.1109/MSP.2018.2875349>
- McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E., & Nieto, O. (2015). Librosa: Audio and music signal analysis in Python. *Proceedings the Python Science Conference*, 18–25. <https://doi.org/10.25080/Majora-7b98e3ed-003>
- Müller, M. (2021). *Fundamentals of music processing – Using Python and Jupyter notebooks* (2nd ed., pp. 1–495) [Monograph]. Springer Verlag. <https://doi.org/10.1007/978-3-030-69808-9>
- Müller, M., McFee, B., & Kinnaird, K. (2021). Interactive learning of signal processing through music: Making Fourier analysis concrete for students. *IEEE Signal Processing Magazine*, 38(3), 73–84. <https://doi.org/10.1109/MSP.2021.3052181>
- Müller, M., & Rosenzweig, S. (2022). PCP notebooks: A preparation course for Python with a focus on signal processing. *Journal of Open Source Education (JOSE)*, 5(47), 148:1–5. <https://doi.org/10.21105/jose.00148>
- Müller, M., & Zalkow, F. (2019). FMP Notebooks: Educational material for teaching and learning fundamentals of music processing. *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 573–580. <https://doi.org/10.5281/zenodo.3527872>

- 196 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin,
197 Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E. Z., DeVito, Z.,
198 Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019).
199 PyTorch: An imperative style, high-performance deep learning library. *Advances in*
200 *Neural Information Processing Systems (NeurIPS)*, 8024–8035.

DRAFT